工程分享：基于 Claude Agent SDK 的零售贷款产品个性化推荐

版本： 1.0
作者： 王巍巍
日期： 2026 年 2 月 8 日

文档修订历史

修订:

日期	参与人	版本	动作	结论
2026/2/8	Charles Wang	1.0	起草文档	N/A

目录

文档修订历史 2

1 什么是 CLAUDE AGENT SDK 3

2 为什么采用 CLAUDE AGENT SDK 构建智能体而非传统方式构建 4

2.1 传统 Agent 构建方式与 Claude Agent SDK 的不同 4
2.2 Claude Agent SDK 的主要优势 4

3 CLAUDE AGENT SDK 的主要能力 5

3.1 核心运行时能力 5
3.2 内置工具能力 5
3.3 平移 Claude Code CLI 的能力 6
3.4 AgentLoop 与上下文能力 6

4 CLAUDE AGENT SDK 的安装和使用 6

4.1 安装方式 6
4.2 Windows 上安装 Claude Agent SDK 的特殊处理 7
4.3 [功能增强]切换到本地已经安装好的 Claude Code CLI 7
4.4 [功能增强]切换到本地 Ollama 运行的 LLM 8

5 演示案例分享：零售贷款产品的个性化推荐 10

5.1 背景介绍 10
5.2 整体架构设计 10
5.3 为什么技术选型 Claude Agent Sdk 11
5.4 系统演示 11
5.5 详细源代码讲解 17
5.5.1 不涉及 Agent 的纯业务 17
5.5.2 Claude Agent SDK 能力-MCP 工具 18
5.5.3 主 Agent-定义人设、工具和主流程 20
5.5.4 Sub-Agent 与 Task 委派 21
5.5.5 业务闭环 23
5.5.6 LLM 除了 Ollama 模型以外，也可以切换到云端模型 25

6 相关资源 25

1 什么是 Claude Agent SDK

Claude Agent SDK 的前身是 Claude Code SDK，它是强大的自主智能体 Agent 构建工具。早期因为框架定位比较窄，只用于编码领域，因此叫 Claude Code SDK，后续发现它在非编码领域同样胜任，所以为了拓宽其应用范围，Anthropic 将其改为 Claude Agent SDK。

Claude Agent SDK 对于 typescript 和 python 主流的语言都有对应的版本。如表所示：

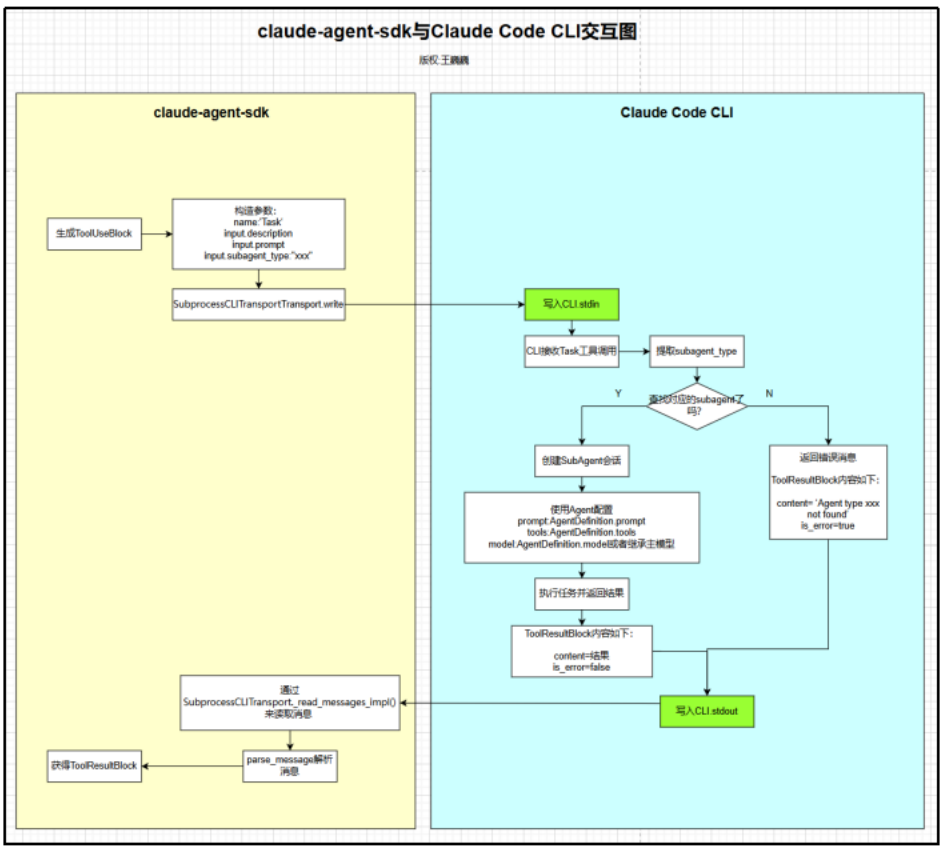
框架名称	协议	Github 首页
claude-agent-sdk-typescript	Anthropic PBC	https://github.com/anthropics/claude-agent-sdk-typescript
claude-agent-sdk-python	MIT	https://github.com/anthropics/claude-agent-sdk-python

另外，Anthropic 官方还出了一个 Demo 工程，https://github.com/anthropics/claude-agent-sdk-demos。

Claude Agent SDK 主要是一套工具的集合，它帮助开发者基于 Claude Code 来构建功能强大的智能体，它将 Claude Code 的核心能力封装为可编程的接口。因此它与 Claude Code CLI 不同，Claude Code CLI 是从命令行交互式工具的角度来使用 Claude 功能，而 Claude Agent SDK 则是为开发者提供的基于代码的集成框架，即将 Claude Code 的能力封装为 API。如表所示，这是 Claude Code CLI 与 Claude Agent SDK 的对比：

对比维度	Claude Code CLI	Claude Agent SDK
定位	交互式的命令行 CLI 工具	基于 Claude Code CLI 的编程接口
面向使用者	开发人员	开发人员 & 程序
是否可以嵌入到应用	不可以	可以

Claude Agent SDK 与 Claude Code CLI 的通信方式如下：



2 为什么采用 Claude Agent SDK 构建智能体而非传统方式构建

2.1 *传统Agent构建方式与Claude Agent SDK的不同*

传统的 Agent 构建方式，是在用 API 自己构建一个智能体引擎，即编排、工具循环、上下文、验证都要自己造。例如需要自己实现完整的 Tool Loop（工具循环），从发 prompt → 拿回复 → 解析是否要调工具 → 执行工具 → 把结果塞回对话 → 再发下一轮……。另外，多轮推理、工具检测与执行、权限、流式输出等都要自己写、自己维护。而状态管理恰恰是最复杂的地方。

而对于 Claude Agent SDK 来说，它内置了 Agent Loop，也就是上述的所有过程均由 SDK 自行完成，这样就可以极大的简化了开发者的负担。开发者只需要定义任务（prompt）并声明使用工具就可以快速的开发智能体，而执行流程、多步推理、工具调用与结果回传均由 SDK 来负责。对应源代码参见：

这两者的区别参见下表：

对比维度	传统自建 Agent 方式	基于 Claude Agent SDK 构建智能体方式
编排逻辑	自己实现 Tool Loop	内置，无需手写循环
多步推理与工具	自己解析、执行、回传	SDK 自动完成
文件/终端/编辑	需自己设计并实现	内置 Read / Edit / Bash 等
上下文与长对话	自己设计压缩、摘要、会话	Sessions + Compaction 等内置
子任务与并行	自己设计子 agent 与通信	内置 Subagents

2.2 *Claude Agent SDK的主要优势*

Claude Agent SDK 的主要优势有以下几点：

1. **不用操心编排逻辑**

传统方式下，每次都要编写“发送请求→解析 tool_calls→执行→拼接 messages→再请求”的逻辑，容易出错且难复用。而 Claud Agent SDK 则将编排逻辑内置，开发人员只需要写“做什么”和“用什么工具”，而不是“怎么循环”，因此开发速度更快，心智负担更小。

2. **内置「收集上下文 → 行动 → 验证」的闭环**

Claude Agent SDK 本身的设计理念就是“收集上下文→行动→验证”的循环。其中收集上下文的主要工作包括文件系统、Agentic search、子智能体、Compaction 等都由 SDK 支持；行动方面内置了 Read、Edit、Bash、Glob、Grep、WebSearch、WebFetch 等能力，并可扩展自定义工具与 MCP；验证主要是可以结合规则来验证或者 LLM-as-judge 做自检。但是传统方式下，这一整套循环通常要自己从零实现并调优。

3. **开箱可用的生产级能力**

Claude Agent SDK 与 Claude Code 同引擎：文件读写、命令执行、代码编辑与校验等能力已经过大规模使用验证。开发者无需从零实现“读文件→跑命令→改代码→再验证”这套基础设施。并且内置会话管理，会话可以跨轮次，可以跨天恢复；支持上下文压缩、支持权限，支持 Hooks，因此更接近“可直接上线”的智能体。

3 Claude Agent SDK 的主要能力

3.1 *核心运行时能力*

Claude Agent SDK 的核心运行时能力如表所示：

能力	说明
Built-in tools（内置工具）	读文件、跑命令、搜网页、改代码等，开箱即用，无需自己实现工具执行逻辑
Hooks（钩子）	在智能体执行流程中插入自定义逻辑，便于监控、审计或改写行为
Subagents（子智能体）	主智能体可派发子任务给独立子智能体，子智能体在独立上下文中运行并返回摘要，实现任务分解与并行。 子智能体的价值主要包括两个方面： ● 并行处理：可启动多个子智能体，同时处理不同任务； ● 上下文管理：子智能体拥有独立的上下文窗口，仅会将相关信息反馈给主智能体，而非传递全部上下文。这一特性使其特别适合处理 “需筛选大量信息、且大部分信息无用” 的任务。
MCP（模型上下文协议）	用标准协议连接外部工具与服务（Slack、GitHub、数据库等），统一鉴权与调用。
Permission（权限）	按工具/操作做细粒度控制（如只读、需确认编辑、禁止危险命令等）
Sessions（会话）	跨多轮、跨天保持会话状态，支持暂停与恢复，适合长任务

3.2 *内置工具能力*

Claude Agent SDK 调用 Claude Code CLI，所以与 Claude Code CLI 同源，内置工具见 Debug 信息：

```
'tools' = {list: 18} ['Task', 'TaskOutput', 'Bash', 'Glob', 'Grep', 'ExitPlanMode', 'Read', 'Edit', 'Write', 'NotebookEdit', 'WebFetch', 'TodoWrite', 'WebSearch', 'TaskStop', 'AskUserQuestion', 'Skill', 'EnterPlanMode', 'To
    00 = {str} 'Task'
    01 = {str} 'TaskOutput'
    02 = {str} 'Bash'
    03 = {str} 'Glob'
    04 = {str} 'Grep'
    05 = {str} 'ExitPlanMode'
    06 = {str} 'Read'
    07 = {str} 'Edit'
    08 = {str} 'Write'
    09 = {str} 'NotebookEdit'
    10 = {str} 'WebFetch'
    11 = {str} 'TodoWrite'
    12 = {str} 'WebSearch'
    13 = {str} 'TaskStop'
    14 = {str} 'AskUserQuestion'
    15 = {str} 'Skill'
    16 = {str} 'EnterPlanMode'
    17 = {str} 'ToolSearch'
```

这些内置工具的详细介绍如表所示：

内置工具名	工具描述
Read	读取工作目录下的文件（代码、配置、文档等）
Write	创建新文件
Edit	对已有文件做精确编辑（修 bug、重构、小改
Bash	执行终端命令与脚本（git、npm、测试、构建等）
Glob	按模式查找文件（如 **/*.ts、src/**/*.py）。尤其是当 Claude 遇到日志、用户上传文件等大型文件时，会通过 grep、tail 等 bash 脚本，决定如何将这些文件加载到上下文当中。
Grep	用正则搜索文件内容（查函数调用、变量使用等）
Web Search	联网搜索（查文档、API、最新信息）
Web Fetch	抓取并解析网页内容
Code execution	执行生成的代码（沙箱内运行）
Text editor	文本编辑能力（与 Read/Edit 等配合）
Memory	持久化记忆与检索，供多轮对话使用
Tool search	在大量工具中按语义搜索并选择要调用的工具

3.3 平移Claude Code CLI的能力

在 Claude Agent SDK 的选项中启用 setting_sources: ['project']之后，可以平移 Claude Code CLI 的能力。

特性	说明	配置位置
Skills	用 Markdown 定义的可复用能力/流程	.claude/skills/SKILL.md
Slash commands	自定义快捷命令（常见任务）	.claude/commands/*.md
Memory	项目级上下文与说明（系统提示/记忆）	CLAUDE.md 或 .claude/CLAUDE.md
Plugins	扩展：自定义命令、子智能体、MCP 服务器等	通过 plugins 选项以代码方式配置

3.4 AgentLoop与上下文能力

Claude Agent SDK 内置 Agent Loop，可自动完成“收集上下文→决策→调用工具→验证→再循环”，无需手写编排循环。同时，在上下文管理方面，Claude Agent SDK 支持压缩机制（接近上下文上限时自动压缩/摘要历史），减少爆窗与重复传长历史。

4 Claude Agent SDK 的安装和使用

4.1 安装方式

需要满足以下条件才能安装：

1). Python 3. 10 或者更高版本

2). Node.js

3). Claude Code 2.0.0 或者更高版本呢

4）申请 Anthropic API 的密钥

运行以下命令 pip install claude-agent-sdk 即可安装 Claude Agent SDK。

4.2 Windows 上安装Claude Agent SDK的特殊处理

如果是 Windows 上安装 Claude Agent SDK，需要把 Git Bash 的 bash.exe 添加到环境变量。如图所示，添加环境变量 CLAUDE_CODE_GIT_BASH_PATH，并且指向本机的 Git Bash 的 base.exe 的位置。

可利用以下测试程序来测试正确性，如果能正确输出，则说明 Claude Agent SDK 可正常工作：

```
import anyio
from claude_agent_sdk import query
async def main():
    async for message in query(prompt="帮我写一个 Python 函数来计算八皇后问题"):
        print(message)
anyio.run(main)
```

4.3 [功能增强]切换到本地已经安装好的Claude Code CLI

默认情况下，Claude Agent SDK 所调用的 Claude Code CLI 并非我们所期望的 Claude Code CLI。而是当前 Claude Code CLI 已经捆绑的版本的 Claude Code CLI。如图所示：

```
> 'mcp_servers' = {list: 0} []
  'model' = {str} 'claude-sonnet-4-5-20250929'
  'permissionMode' = {str} 'default'
> 'slash_commands' = {list: 10} ['debug', 'compact', 'context', 'cost', 'init', 'pr-comments', 'release-notes', 'review', 'security-review', 'insights']
  'apiKeySource' = {str} 'none'
  'claude_code_version' = {str} '2.1.33'
  'output_style' = {str} 'default'
> 'agents' = {list: 5} ['Bash', 'general-purpose', 'statusline-setup', 'Explore', 'Plan']
> 'skills' = {list: 1} ['debug']
> 'plugins' = {list: 0} []
  'uuid' = {str} '9eabc3c6-eeed-4fb7-9ec3-badb596b132b'
  __len__ = {int} 16
```

这里的 v2.1.33 版本的 Claude Code 并非我们所期望已经安装的 Claude Code CLI 版本。而是 Claude Agent SDK 自动捆绑的 Claude Code CLI 的版本。如果希望采用系统级安装版本的 Claude Code CLI，可以用 ClaudeAgentOptions(cli_path="claude code cli 的路径")来传入。

cli_path 获得的具体方法如下：

1. 先安装好 Claude Code CLI。

2. 在 Powershell 中运行以下命令来获得 cli_path: Get-Command claude | Format-List Source

```
Windows PowerShell
版权所有 (C) Microsoft Corporation。保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

加载个人及系统配置文件用了 1237 毫秒。
(base) PS C:\Users\Charles> Get-Command claude | Format-List Source


Source : C:\Users\Charles\.local\bin\claude.exe
```

获得了 cli_path 之后，将其填入 ClaudeAgentOptions 即可。因此原先的代码改为即可：

```
import anyio
from pathlib import Path
from claude_agent_sdk import query, ClaudeAgentOptions

# 把下面改成你本机查到的路径（Get-Command claude 命令获得的路径）
CLAUDE_CLI = r"C:\Users\Charles\.local\bin\claude.exe"

async def main():
    options = ClaudeAgentOptions(cli_path=CLAUDE_CLI)
    async for message in query(prompt="帮我写一个 Python 函数来计算八皇后问题?", options=options):
        print(message)

anyio.run(main)
```

4.4 [功能增强]切换到本地Ollama运行的LLM

默认情况下，采用 Claude Code CLI 中用到的付费的 LLM。也可以切换到本地 Ollama 运行的免费 LLM。注意，Ollama 的版本必须是 v0.14 以上，因为 Anthropic 兼容的（/v1/messages）是 v0.14 才有的。版本过低时，

连接/v1/messages 会报 404 错误。

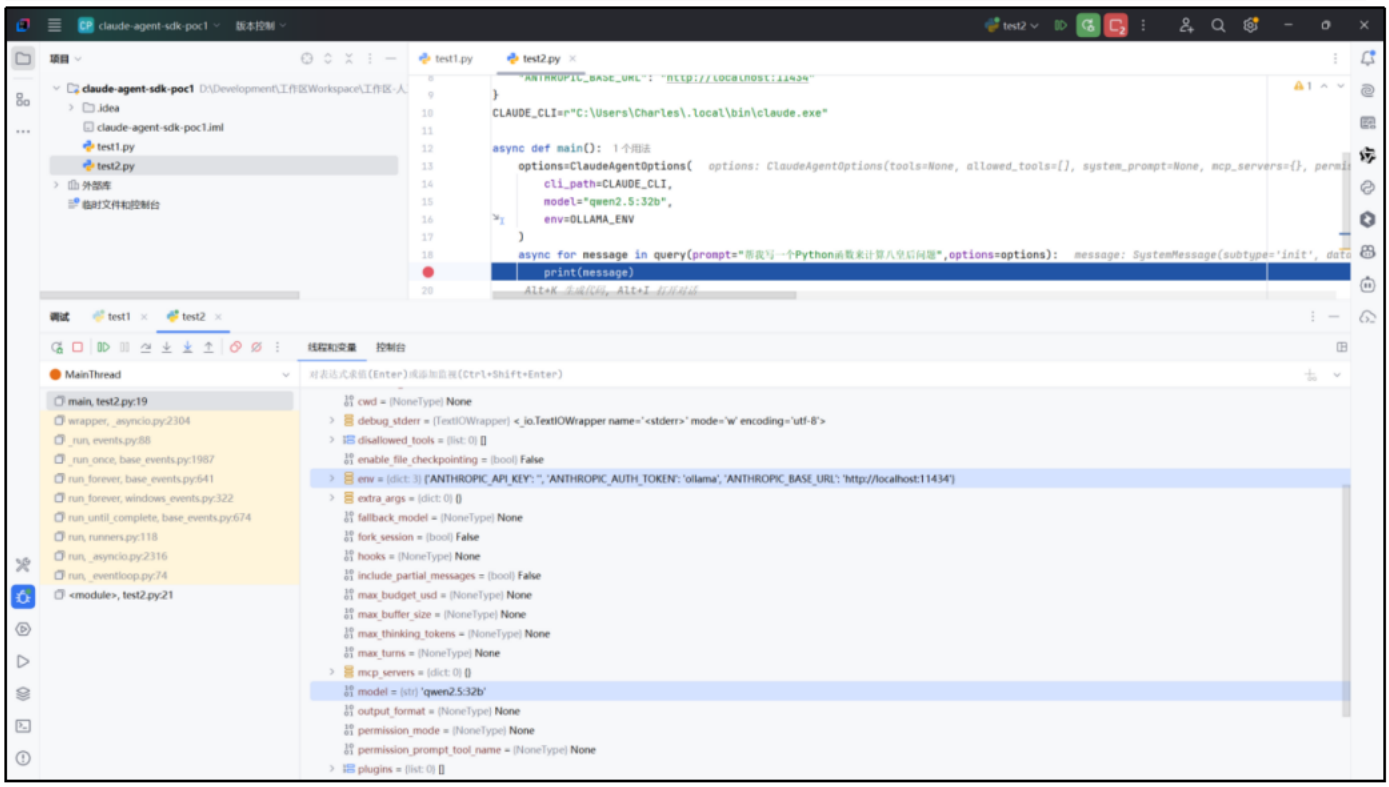
可以通过在终端运行 ollama -v 命令来查看 ollama 的版本。版本过低则需先升级到 v0.14 以上。

这里以本地 ollama 启动的 qwen2.5:32b 为例，改造程序如下，重点关注黑体字部分：

1. 需要配置 OLLAMA_ENV 环境变量组信息，它包括 ANTHROPIC_AUTH_TOKEN 为 ollama ,ANTHROPIC_API_KEY 为空字符串, ANTHROPIC_BASE_URL 指向本地 ollama 的 URL。

2. Claude AgentOptions 中新增两个选项，一个选项是 env 配置为 OLLAMA_ENV，另一个选项 model 配置为本地要用的 LLM，例如 qwen2.5:32b:

```
import anyio
from claude_agent_sdk import query,ClaudeAgentOptions
# 让 Claude Code CLI 使用 Ollama 后端
OLLAMA_ENV = {
    "ANTHROPIC_AUTH_TOKEN": "ollama",
    "ANTHROPIC_API_KEY": "",
    "ANTHROPIC_BASE_URL": "http://localhost:11434"
}
CLAUDE_CLI=r"C:\Users\Charles\.local\bin\claude.exe"
async def main():
    options=ClaudeAgentOptions(
        cli_path=CLAUDE_CLI,
        model="qwen2.5:32b",
        env=OLLAMA_ENV
    )
    async for message in query(prompt="帮我写一个 Python 函数来计算八皇后问题",options=options):
        print(message)
anyio.run(main)
```



这样即可正常工作，如图所示，用 Python 最终实现了八皇后问题。

5 演示案例分享：零售贷款产品的个性化推荐

5.1 背景介绍

这是一个针对零售贷款场景的演示项目。有客户有产品，需要先加载客户数据和产品数据，并依次训练意愿模型和推荐模型，并对指定客户进行个性化的推荐以及意愿预测。

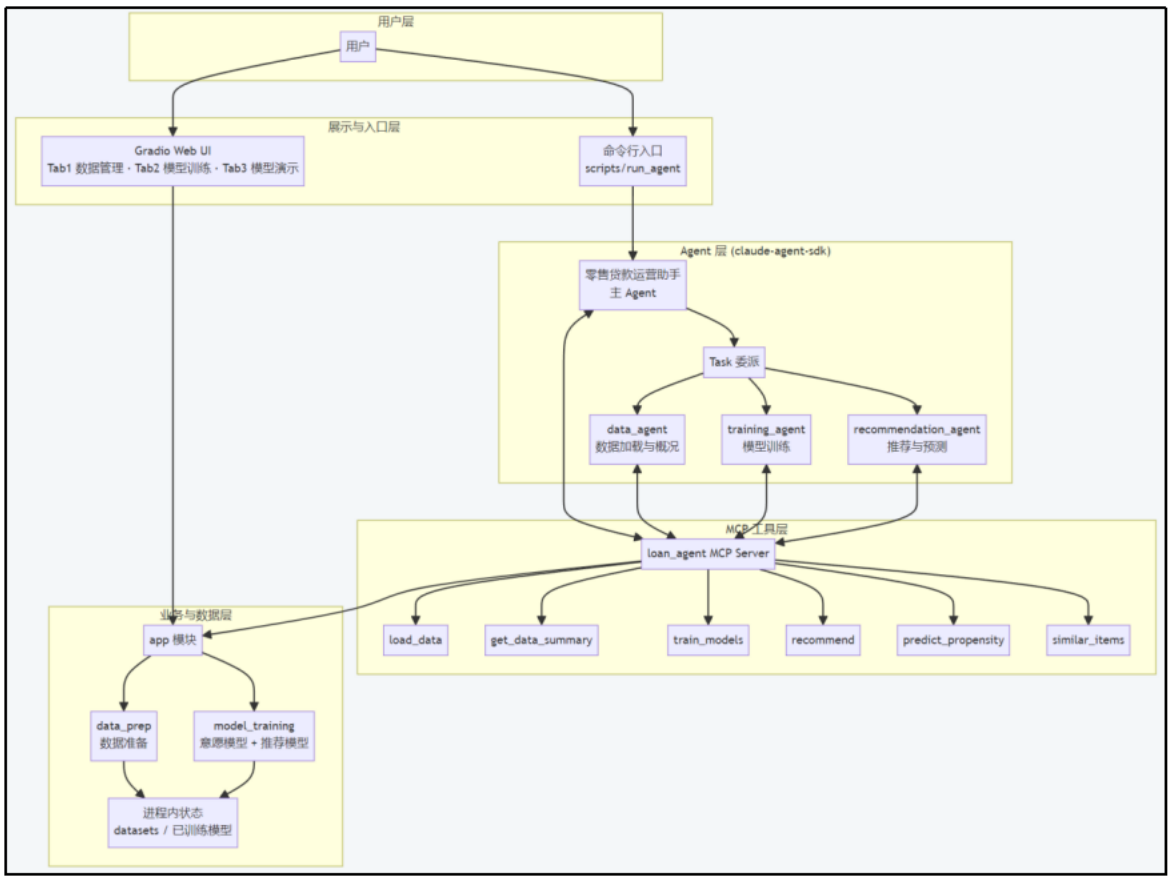
5.2 整体架构设计

整体架构设计如下。

1) 展示与入口层通过 Gradio Web UI 构建。它包含多个 Tab 页，有数据管理、模型训练、模型演示和智能对话。

2) Agent 层是基于 claude-agent-sdk 的多 Agent 架构。主 Agent 会把任务委派给 3 个 sub-agent，分别是 data_agent 负责数据加载和概况、training_agent 负责模型训练、recommendation_agent 负责推荐与预测。

3) 这几个 subagent 都会调用 MCP 工具层上的暴露出的 tool 来完成相关任务。

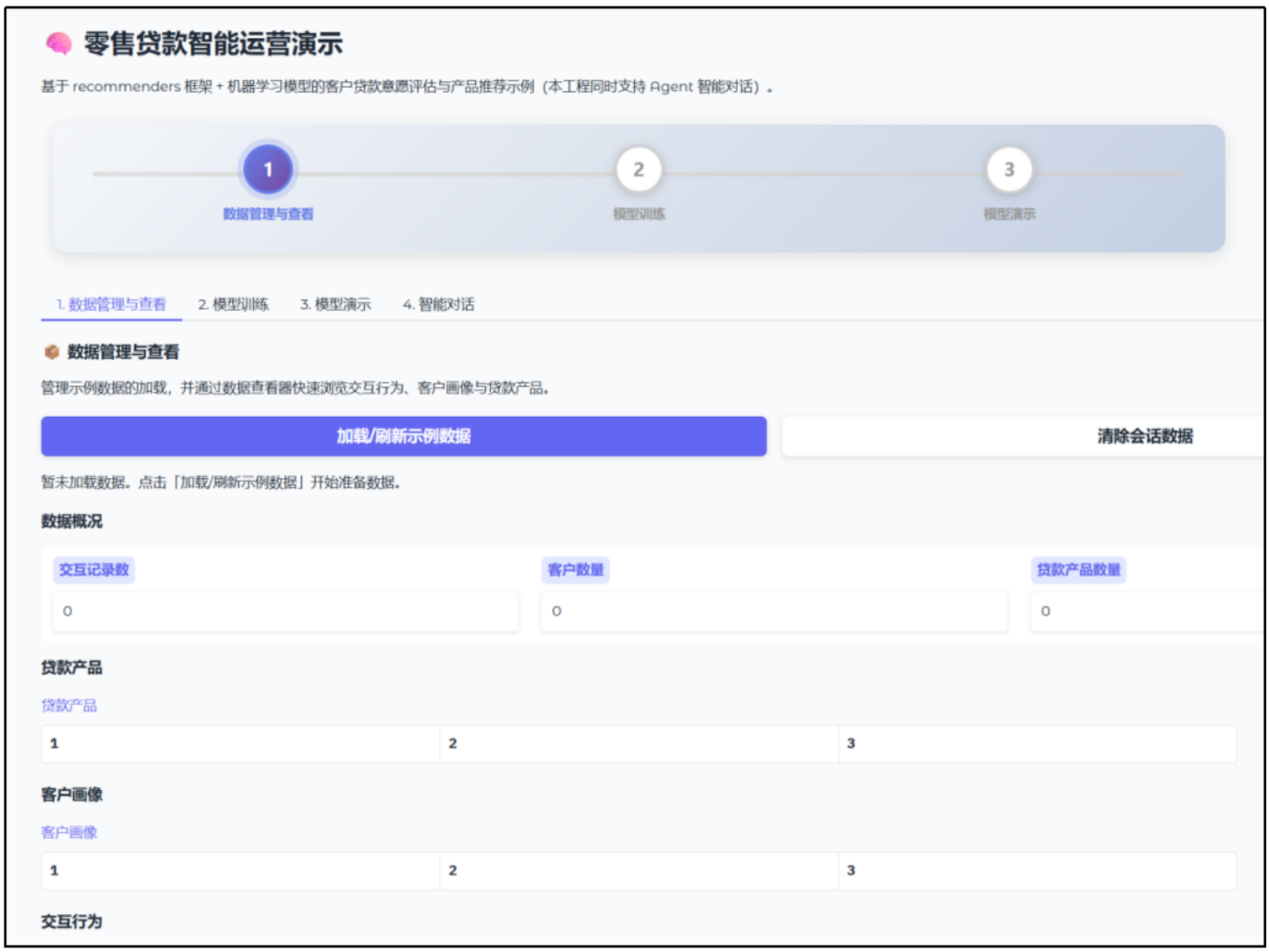


5.3 *为什么技术选型Claude Agent Sdk*

本系统需要完成“加载数据”→“训练模型”→“推荐/预测”的完整流水线，并希望主 Agent 能按任务类型把工作委派给专门的数据、训练、推荐子 Agent。Claude Agent Sdk 提供现成的 Agent 编排、MCP 工具协议、Task 子 Agent 委派以及多模型后端（云端/本地）支持，用少量 API 即可把 6 个业务能力封装成 MCP 工具、定义三个职责清晰的子 Agent，并保持同一套逻辑在不同环境下运行，因此选择 Claude Agent Sdk 作为智能体与工具链的实现基础。

5.4 *系统演示*

运行 main.py 并访问 http://localhost:7860 进入首页，可以看到有三个步骤：



点击“加载/刷新示例数据”，可以看到它从 MovieLens 数据集加载对应的数据，并且映射为贷款产品标签与风险分数等信息。贷款产品如下：

数据已加载。交互记录: 100,000 条，客户数: 943，贷款产品数: 1,682。

数据概况

交互记录数	客户数量	贷款产品数量
100000	943	1682

贷款产品

贷款产品

item_id	title	genres	product_tags	item_risk_score	item_risk_label
51	零售贷款产品 0004	["Documentary", "Drama"]	["政策支持贷", "综合经营贷"]	0.275	低风险偏好
346	零售贷款产品 0005	["Fantasy", "Film-Noir"]	["创新项目贷", "高收益专项贷"]	0.675	高风险偏好
474	零售贷款产品 0006	["Horror", "Musical"]	["文化旅游贷", "高风险特色贷"]	0.65	高风险偏好
265	零售贷款产品 0007	["Mystery", "Romance"]	["婚庆装修贷", "调研定制贷"]	0.47500000000000003	中等风险偏好
465	零售贷款产品 0008	["Sci-Fi", "Thriller"]	["应急备用贷", "科技研发贷"]	0.6499999999999999	高风险偏好
451	零售贷款产品 0009	["War", "Western"]	["传统实业贷", "制造业升级贷"]	0.7	高风险偏好
86	零售贷款产品 0010	["Action", "Adventure"]	["创业扩张贷", "高额度周转贷"]	0.7	高风险偏好
257	零售贷款产品 0011	["Animation", "Children's"]	["家庭消费贷", "教育分期贷"]	0.325	低风险偏好
1014	零售贷款产品 0012	["Comedy", "Crime"]	["生活消费贷", "风控重点产品"]	0.5	中等风险偏好
222	零售贷款产品 0013	["Documentary", "Drama"]	["政策支持贷", "综合经营贷"]	0.275	低风险偏好
40	零售贷款产品 0014	["Fantasy", "Film-Noir"]	["创新项目贷", "高收益专项贷"]	0.675	高风险偏好
29	零售贷款产品 0015	["Horror", "Musical"]	["文化旅游贷", "高风险特色贷"]	0.65	高风险偏好
785	零售贷款产品 0016	["Mystery", "Romance"]	["婚庆装修贷", "调研定制贷"]	0.47500000000000003	中等风险偏好

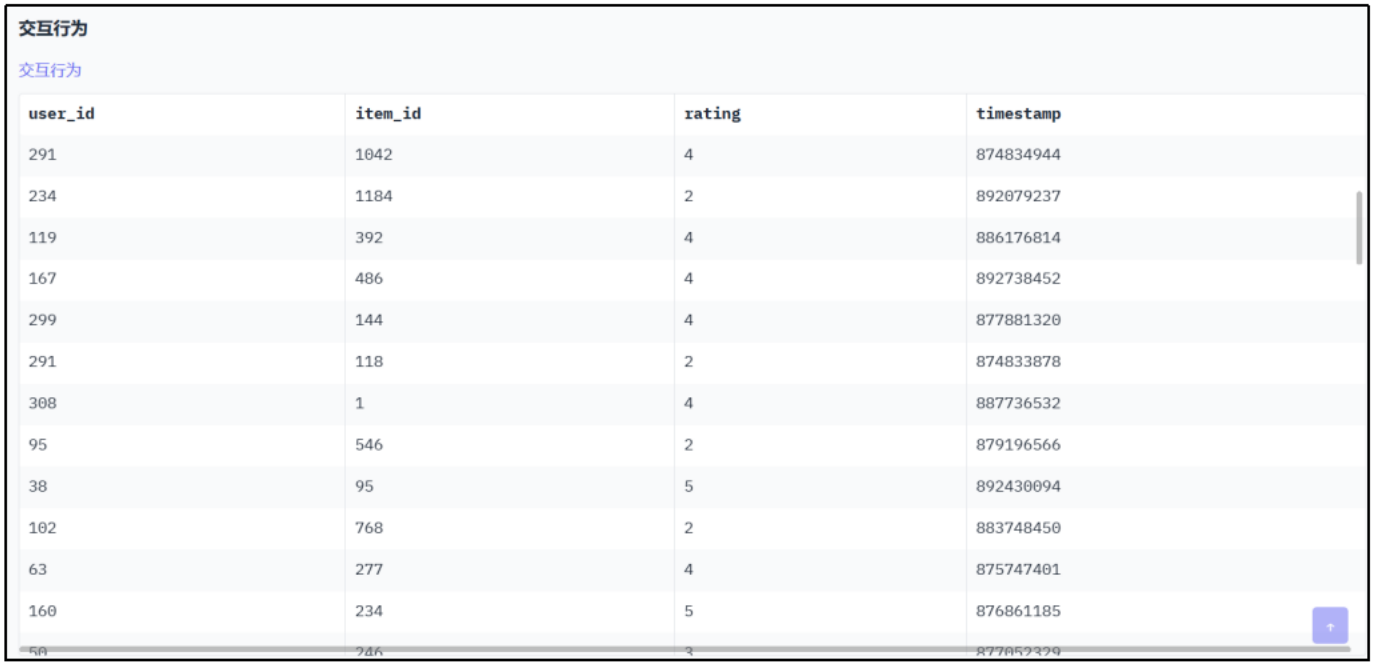
客户画像数据如下：

客户画像

客户画像

user_id	top_tags	user_risk_score	user_risk_label	asset_label	asset_score
1	["家庭消费贷", "教育分期贷", "创新项目贷"]	0.5474264705882352	中等风险偏好	稳健型客户	0.55
2	["婚庆装修贷", "调研定制贷", "创业扩张贷"]	0.5741935483870969	中等风险偏好	成长型客户	0.35
3	["创新项目贷", "高收益专项贷", "创业扩张贷"]	0.6037037037037037	高风险偏好	稳健型客户	0.55
4	["创业扩张贷", "高额度周转贷", "创新项目贷"]	0.6375000000000001	高风险偏好	稳健型客户	0.55
5	["婚庆装修贷", "调研定制贷", "家庭消费贷"]	0.5274285714285715	中等风险偏好	高潜力客户	0.8
6	["文化旅游贷", "高风险特色贷", "创业扩张贷"]	0.5561611374407582	中等风险偏好	成长型客户	0.35
7	["家庭消费贷", "教育分期贷", "文化旅游贷"]	0.5475186104218364	中等风险偏好	成长型客户	0.35
8	["创新项目贷", "高收益专项贷", "家庭消费贷"]	0.5368644067796611	中等风险偏好	稳健型客户	0.55
9	["婚庆装修贷", "调研定制贷", "创业扩张贷"]	0.5602272727272726	中等风险偏好	高潜力客户	0.8
10	["婚庆装修贷", "调研定制贷", "家庭消费贷"]	0.5388586956521739	中等风险偏好	稳健型客户	0.55
11	["政策支持贷", "综合经营贷", "生活消费贷"]	0.5386740331491713	中等风险偏好	成长型客户	0.35
12	["生活消费贷", "风控重点产品", "传统实业贷"]	0.5524509803921569	中等风险偏好	高潜力客户	0.8
13	["应急备用贷", "科技研发贷", "政策支持贷"]	0.5517688679245283	中等风险偏好	成长型客户	0.35

交互行为数据如下：



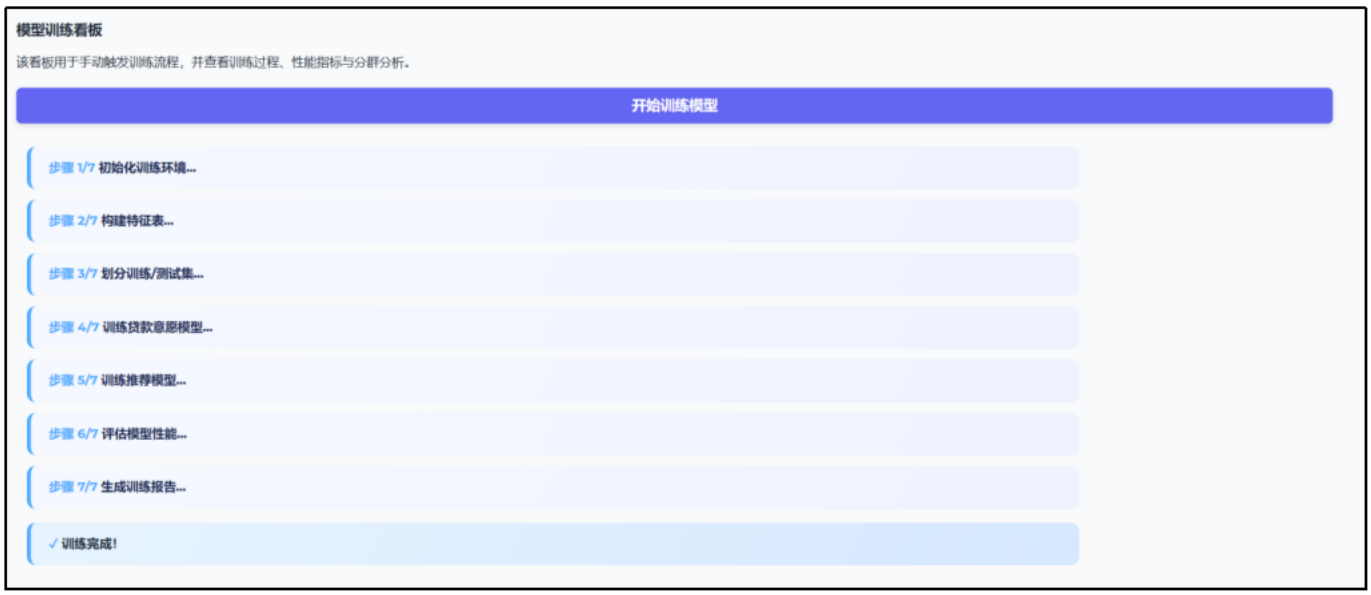
交互行为

交互行为

user_id	item_id	rating	timestamp
291	1042	4	874834944
234	1184	2	892079237
119	392	4	886176814
167	486	4	892738452
299	144	4	877881320
291	118	2	874833878
308	1	4	887736532
95	546	2	879196566
38	95	5	892430094
102	768	2	883748450
63	277	4	875747401
160	234	5	876861185
50	246	3	877052329

点击“下一步：模型训练”按钮进入模型训练步骤：

点击“开始训练模型”按钮进行模型训练，上方面板可以看到模型训练的步骤：



下方分 2 个子 Tab 可以看到模型训练的细节。例如贷款意愿模型的训练细节：

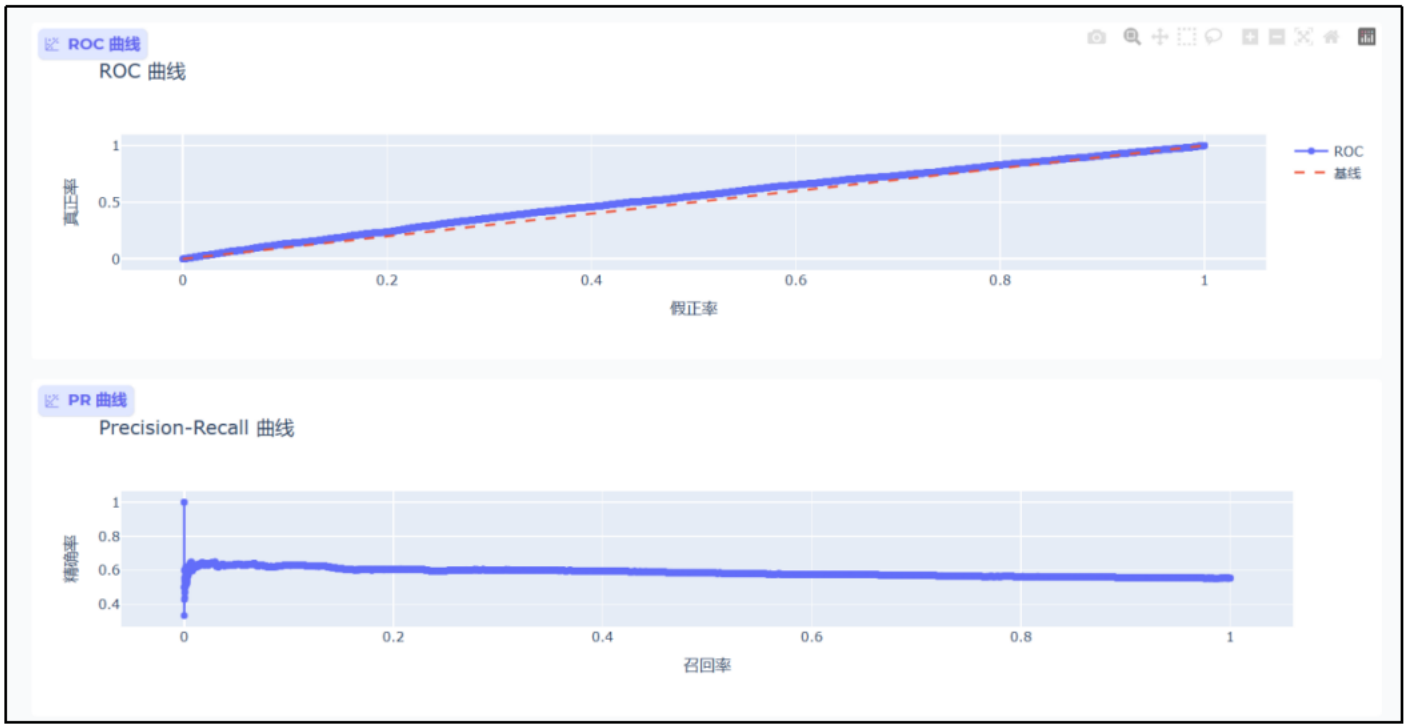
贷款意愿模型　　推荐模型

1. **构建特征表**：结合交互、用户画像与产品画像生成 100000 条样本
2. **划分训练/测试集**：按 8:2 划分，随机种子 42，保持标签分布稳定
3. **特征预处理**：数值特征标准化，类别特征进行 One-Hot 编码（将分类变量拆分为0/1特征）
4. **训练逻辑回归模型**：使用 lbfgs 求解器（拟牛顿法，收敛更稳定），最大迭代次数 500
5. **性能评估**：ROC-AUC（衡量模型区分正负样本能力）=0.538，并生成分类报告

- **样本总数**: 100000
- **训练集大小**: 80000
- **测试集大小**: 20000
- **正样本占比**: 55.4%
- **训练集正样本占比**: 55.4%

- **准确率**: 55.4%
- **精确率**: 55.4%
- **召回率**: 99.9%
- **F1 分数**: 71.2%

贷款意愿模型训练时的 ROC 曲线和 PR 曲线：



阈值对比、混淆矩阵、分群指标和关键特征权重如下：

阈值对比

threshold	precision	recall	accuracy	f1
0.3	0.55375	1	0.55375	0.7127916331456154
0.4	0.55375	1	0.55375	0.7127916331456154
0.5	0.5536921151439299	0.9986455981941309	0.5535	0.7123993558776167
0.6	0.6292765002804263	0.10130925507900677	0.4693	0.17452169855342978
0.7	0	0	0.44625	0

混淆矩阵

预测负类	预测正类
10	8915
15	11060

分群指标

分群维度	分群标签	样本数	平均概率	准确率	精确率	召回率	F1 分数
客户层级	成长型客户	7037	0.5455649229417976	0.5425607503197385	0.5430690245293782	0.9960763798064347	0.7029072450392249
客户层级	稳健型客户	5906	0.5639441618289815	0.5724686759227904	0.5724686759227904	1	0.728114568752019
客户层级	高潜力客户	7057	0.5535518337974256	0.5485333711208729	0.5485333711208729	1	0.7084553440702781
风险偏好	中等风险偏好	19491	0.55416974395665	0.5534349186804166	0.5534633145202668	0.99990730441231	0.7125305502344936
风险偏好	高风险偏好	509	0.5400538195535612	0.555992141453831	0.5628865979381443	0.9512195121951219	0.7072538860103628

关键特征权重

特征	系数权重
低风险贷款产品	0.15772653142684612
需求匹配标签数量	0.09560287019558865
稳健型客户身份	0.0795469862674142
高风险贷款产品	-0.07690904254624344
高风险偏好客户	0.06362827653573386
中等风险偏好客户	0.05684888528341137

推荐模型的模型训练细节如下：

点击下一步“模型演示”，进入模型演示界面，并点击“获取推荐”按钮。则可以看到为该客户推荐的贷款商品清单。同时在右边可以显示出该客户的用户画像。

5.5 *详细源代码讲解*

5.5.1 不涉及 Agent 的纯业务

数据准备入口位于 app/data_prep.py。它按规模加载 MovieLens 交互与物品，再生成客户画像与贷款产品画像，返回三元组供后续训练与推荐使用。MovieLens 是一个经典的电影评分数据集，由美国明尼苏达大学 GroupLens 研究组维护，常用来做推荐系统、协同过滤的研究和教学。它的主页是 https://grouplens.org/datasets/movielens/。本工程通过 recommenders 库在首次加载时自动拉取 100k 数据。对应的源代码如下：

```
def prepare_datasets(size: str = "100k") -> Tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """准备交互数据、客户画像与贷款产品画像。"""
    interactions = load_interactions(size=size)
    items = load_items(size=size)
    item_profiles = build_item_profiles(items)
    user_profiles = build_user_profiles(interactions, item_profiles)
    return interactions, user_profiles, item_profiles
```

因为涉及到的场景是贷款场景，所以把该数据集做了映射，将 MovieLens 的影片类型映射为贷款产品标签与风险分数，用于构建“客户-产品”特征与意愿标签，对应源代码也在 app/data_prep.py 中：

```
GENRE_TO_PRODUCT_TAG = {
    "Action": "高额度周转贷",
    "Adventure": "创业扩张贷",
    # ... 其他类型 → 贷款产品标签
}
GENRE_TO_RISK_SCORE = { "Documentary": 0.2, "Children's": 0.25, ... }
```

意愿模型训练的源代码位于 app/model_training.py 中，它用交互与画像构建特征表、划分训练/测试集，经预处理后训练逻辑回归，封装为带 ROC-AUC 与可解释性的 PropensityModel。

```
def train_propensity_model(
    interactions: pd.DataFrame, user_profiles: pd.DataFrame, item_profiles: pd.DataFrame
) -> PropensityModel:
    features = _build_feature_frame(interactions, user_profiles, item_profiles)
    # ... 特征列、train_test_split、Pipeline(OneHotEncoder+StandardScaler+LogisticRegression)
    return PropensityModel(pipeline=..., feature_names=..., roc_auc=..., ...)
```

推荐模型训练的源代码也位于 app/model_training.py 中，它使用 SAR（协同过滤）在交互表上拟合，得到可做 Top-K 推荐与相似产品的 RecommendationModel。

```
def train_recommendation_model(interactions: pd.DataFrame) -> RecommendationModel:
    model = SAR(
        col_user=USER_COL, col_item=ITEM_COL, col_rating=RATING_COL, col_timestamp=TIMESTAMP_COL,
```

```
        similarity_type="jaccard",
    )
    model.fit(interactions)
    return RecommendationModel(model=model, training_summary=...)
```

推荐/相似产品/预测也位于 app/model_training.py 中，其中推荐模型对外提供 recommend（按用户推荐）、similar_items（按物品找相似）；意愿模型提供 predict_proba（客户-产品意愿概率）。

```
class RecommendationModel:
    def recommend(self, user_id: str, top_k: int = 5) -> pd.DataFrame:
        return self.model.recommend_k_items(...)
    def similar_items(self, item_id: str, top_k: int = 5) -> pd.DataFrame: ...
class PropensityModel:
    def predict_proba(self, features: pd.DataFrame) -> np.ndarray:
        return self.pipeline.predict_proba(features)[:, 1]
```

5.5.2 Claude Agent SDK 能力-MCP 工具

基于架构原则，Agent 不能直接改数据库或调 Python 函数，必须通过“工具”。本工程把 6 个业务动作暴露为 6 个 MCP 工具，由 SDK 的 MCP 服务统一挂载。

六个工具以及业务含义分别如下：

工具	业务含义	对应业务层
load_data	加载示例数据	data_prep.prepare_datasets
get_data_summary	查看当前数据概况	读状态 + 统计
train_models	训练意愿模型 + 推荐模型	model_training
recommend	对某客户做个性化推荐	推荐模型 + 状态
predict_propensity	预测客户对某产品的意愿概率	意愿模型
similar_items	与某产品相似的产品	推荐/相似度

Tools 的定义都在 tools/load_agent_tools.py 中，这里定义了 6 个以 tool 前缀开头的方法。

```
async def tool_load_data(args: dict) -> dict:
    """加载示例数据（MovieLens 100k 转贷款场景）。无需参数。"""
    try:
        datasets = _ensure_datasets()
        interactions, user_profiles, item_profiles = datasets
        n_inter = len(interactions)
        n_users = user_profiles["user_id"].nunique()
```

```
        n_items = item_profiles["item_id"].nunique()
        msg = (
            f"数据已加载。交互记录: {n_inter:,} 条，客户数: {n_users:,}，贷款产品数: {n_items:,}。"
            " 可继续使用「训练模型」或「推荐」等工具。"
        )
        return {"content": _text_content(msg)}
    except Exception as e:
        return {"content": _text_content(f"加载数据失败: {e}")}
    ....
async def tool_recommend(args: dict) -> dict:
    """为指定客户做个性化贷款推荐。参数: user_id（客户 ID，字符串），top_k（推荐数量，默认 5）。"""
    ...
```

大家可能发现，这里都是普通的函数，并没有用类似@MCPTool 注解做声明，那么它们是如何起作用的呢？其实主要是通过 Claude Agent SDK 的 tool()函数把普通的异步函数包装为带元数据的工具，再一次性交给 create_sdk_mcp_server 组成 MCP 服务，供 Runner 挂载。对应的实现位于 tools/load_agent_tool.py 中：

```
from claude_agent_sdk import tool, create_sdk_mcp_server
load_data_tool = tool(
    "load_data",
    "加载示例贷款场景数据（MovieLens 100k 转贷款）。无参数。",
    {},
)(tool_load_data)
...
# train_tool, summary_tool, recommend_tool, predict_tool, similar_tool 同理
...
return create_sdk_mcp_server(
    name="loan_agent",
    version="1.0.0",
    tools=[load_data_tool, train_tool, summary_tool, recommend_tool, predict_tool, similar_tool],
)
```

ClaudeAgentOptions.allowed_tools 需要的是「mcp__服务名__工具名」格式，所以在 tools/loan_agent_tools.py 的 get_loan_agent_allowed_tools()方法中，集中返回 6 个名称，Runner 与主/子 Agent 配置都会用到。

```
def get_loan_agent_allowed_tools() -> list[str]:
    return [
        "mcp__loan_agent__load_data",
        "mcp__loan_agent__train_models",
        "mcp__loan_agent__get_data_summary",
        "mcp__loan_agent__recommend",
        "mcp__loan_agent__predict_propensity",
        "mcp__loan_agent__similar_items",
    ]
```

最后挂在 MCP 到主 Agent 上，它的源代码位于 agents/runner.py。 创建 loan_agent 的 MCP 服务实例后，以字典形式塞入 options_dict["mcp_servers"]，SDK 在会话中会按工具名路由到该服务。

```
mcp_server = build_loan_agent_mcp_server()
# ...
if mcp_server is not None:
    options_dict["mcp_servers"] = {"loan_agent": mcp_server}
```

5.5.3 主 Agent-定义人设、工具和主流程

如前面提到，主 Agent（零售贷款运营助手）主要负责理解用户意图、决定先做什么后做什么、调用哪些工具或委派给谁，最后汇总成自然语言回复。当配置主 Agent 时，Claude Agent SDK 需要配置其 system_prompt(系统提示词）、allowed_tools(允许的工具)、mcp_servers(挂载 loan_agent)这个 MCP 服务。

主 Agent 的人设与工具/Task 约定定义在 agents/definitions.py 中，它会给主 Agent 的 system 文本：限定角色、列出两种完成方式（直接 MCP 或 Task 委派）、约定 Task 参数与三子 Agent 的职责，引导模型按流程选工具或委派。

```
MAIN_SYSTEM_PROMPT = """你是零售贷款智能运营助手。所有回复必须使用中文。
你可以通过两种方式完成用户请求：
【方式一】直接使用 MCP 工具：load_data、get_data_summary、train_models、recommend、predict_propensity、similar_items。
【方式二】通过 Task 调用子 Agent（适合多步骤或需要专项分析的场景）。调用 Task 时必须传入：
- subagent_type：取 data_agent、training_agent、recommendation_agent 之一
- description：简短任务描述（中文）
- prompt：发给该子 Agent 的具体指令（中文）
子 Agent 说明：
- data_agent：负责数据加载与概况查询。...
- training_agent：负责模型训练。...
- recommendation_agent：负责推荐与意愿预测。...
工作流程建议：用户说「加载数据」时可直接调用 load_data 或 Task(data_agent)；..."""
```

allowed_tools 定义在 agents/runner.py 中。主 Agent 可用工具 = SDK 内置的 Read/Grep/Glob/Bash + Task（委派子 Agent）+ 本 MCP 的 6 个业务工具名。

```
allowed = ["Read", "Grep", "Glob", "Bash", "Task"] + get_loan_agent_allowed_tools()
```

然后组装 ClaudeAgentOptions。即把模型参数、system_prompt、allowed_tools、disallowed_tools、mcp_servers、agents 等拼成 options_dict，再传给 ClaudeAgentOptions 构造 SDK 所需配置。见 agents/runner.py。

```
options_dict = {
    **agent_kw,
    "system_prompt": MAIN_SYSTEM_PROMPT,
    "allowed_tools": allowed,
    "disallowed_tools": ["Write", "Edit"],
}
```

```
if mcp_server is not None:
    options_dict["mcp_servers"] = {"loan_agent": mcp_server}
if sub_agents:
    options_dict["agents"] = sub_agents
options = ClaudeAgentOptions(**options_dict)
```

因为我们期望更好的用户体验，所以采用流式驱动。创建 SDK 客户端后先 connect，再 query 用户消息，最后通过 receive_response() 异步迭代每条消息并 yield 给上层，实现流式输出。具体代码也在 agents/runner.py 中。

```
async with ClaudeSDKClient(options=options) as client:
    await client.connect()
    await client.query(user_message, session_id="default")
    async for message in client.receive_response():
        yield message
```

5.5.4 Sub-Agent 与 Task 委派

前面已经讲解了为什么要子 Agent，因为它有 2 个重大价值。第一个价值是职责分离。因为数据、训练、推荐三类操作逻辑不同，主 Agent 如果想“全知”容易冗长且易错，而拆为三个子 Agent，可以让子 Agent 各自只看到自己那部分工具，意图更清晰；第二个价值是上下文隔离。子 Agent 在独立上下文中执行，只收到主 Agent 通过 Task 传来的 subagent_type、description、prompt，避免主对话被大量中间步骤塞满。

三个 Sub-Agent 与业务的对应关系如表所示：

子 Agent	职责	可用 MCP 工具	业务闭环中的位置
data_agent	数据加载与概况查询	load_data、get_data_summary	数据闭环
training_agent	模型训练	train_models	训练闭环
recommendation_agent	推荐与意愿预测	recommend、predict_propensity、similar_items	使用闭环

每个 Sub-Agent 都通过 AgentDefinition 来定义，并且都定义在 agents/definitions.py 中：

数据 Sub-Agent 定义如下。其中 description 与 prompt 限定其只做数据加载与概况查询，tools 仅开放 load_data 与 get_data_summary 这 2 个 MCP Tool。

```
from claude_agent_sdk import AgentDefinition
return {
    "data_agent": AgentDefinition(
        description="负责数据加载与数据概况查询。用于执行加载示例数据、查看当前交互数/客户数/产品数等。",
        prompt="""你是零售贷款场景的数据助手。你只能使用 load_data、get_data_summary 两个工具。
用户可能要求：加载数据、查看数据概况、看看当前有多少条数据等。请根据请求调用相应工具，并用中文简要汇总结果。""",
        tools=[
            "mcp__loan_agent__load_data",
            "mcp__loan_agent__get_data_summary",
        ],
```

```
        model="sonnet",
    ),
```

训练 Sub-Agent 只开放 train_models 这个 MCP Tool，prompt 约定其调用后汇总 ROC-AUC、准确率等指标并用中文回复。

```
"training_agent": AgentDefinition(
    description="负责贷款意愿模型与推荐模型的训练。用于执行训练流程并汇报指标。",
    prompt="""你是零售贷款场景的模型训练助手。你只能使用 train_models 工具。...""",
    tools=["mcp__loan_agent__train_models"],
    model="sonnet",
),
```

推荐 Sub-Agent 开放了 recommend、predict_propensity、similar_items 这 3 个 MCP Tool，它负责在拿到 user_id/item_id 后执行推荐或预测并汇总结果。

```
"recommendation_agent": AgentDefinition(
    description="负责个性化推荐、贷款意愿预测、相似产品查询。需要 user_id 或 item_id 时由主 Agent 或用户提供。",
    prompt="""你是零售贷款场景的推荐与预测助手。你只能使用 recommend、predict_propensity、similar_items 三个工具。...""",
    tools=[
        "mcp__loan_agent__recommend",
        "mcp__loan_agent__predict_propensity",
        "mcp__loan_agent__similar_items",
    ],
    model="sonnet",
),
```

定义完毕这些 Sub-Agent 之后，需要将他们注入到 Claude Agent SDK。其实现逻辑定义在 agents/runner.py。主要是将 build_sub_agent_definitions() 返回的字典赋给 options_dict["agents"]，SDK 在遇到 Task 调用时会按 subagent_type 路由到对应子 Agent。

```
sub_agents = build_sub_agent_definitions()
# ...
if sub_agents:
    options_dict["agents"] = sub_agents
```

为了让主 Agent 能顺利使用 Sub-Agent，在主 Agent 的 SYSTEM_PROMPT 中需要指定。见【方式二】通过 Task 调用子 Agent。且调用 Task 时必须传入三个字段：subagent_type 指定子 Agent，description 与 prompt 描述任务内容，供 SDK 路由并下发指令。

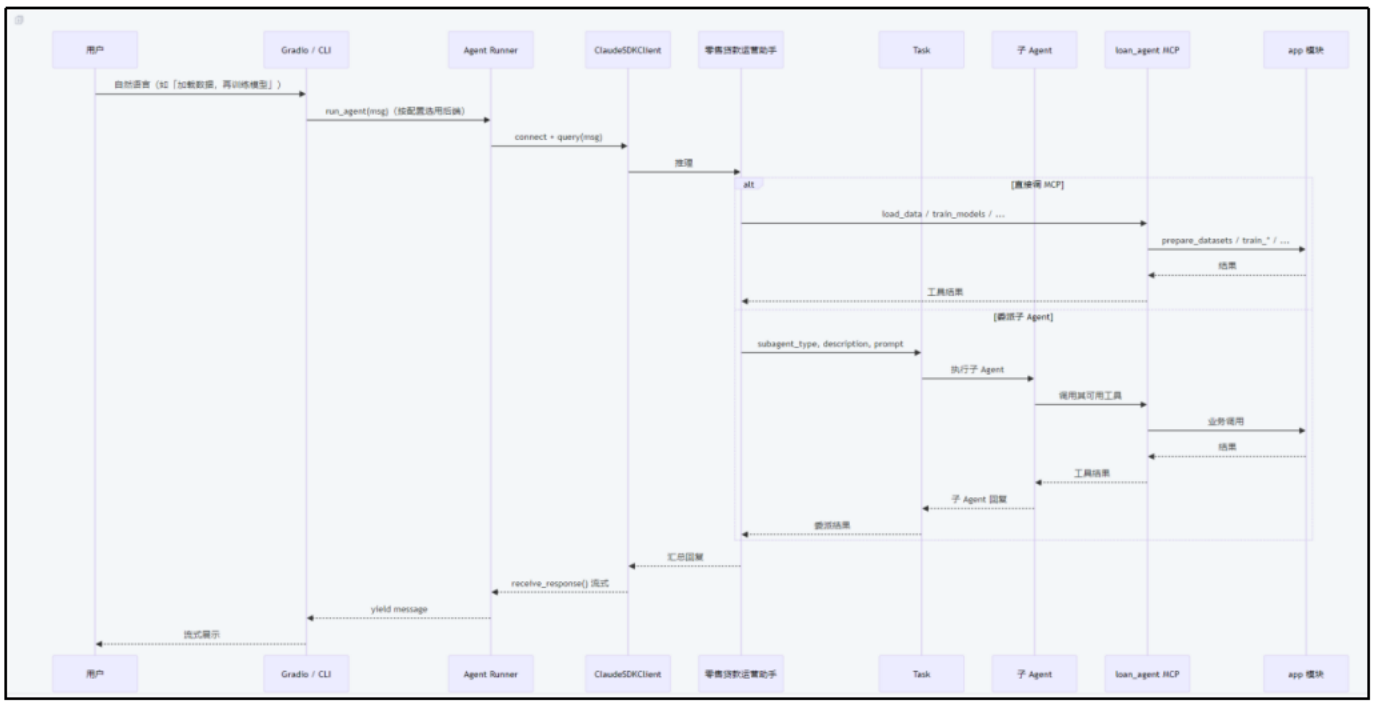
```
MAIN_SYSTEM_PROMPT = """你是零售贷款智能运营助手。所有回复必须使用中文。
你可以通过两种方式完成用户请求：
【方式一】直接使用 MCP 工具：load_data、get_data_summary、train_models、recommend、predict_propensity、similar_items。
【方式二】通过 Task 调用子 Agent（适合多步骤或需要专项分析的场景）。调用 Task 时必须传入：
```

```
- subagent_type：取 data_agent、training_agent、recommendation_agent 之一
- description：简短任务描述（中文）
- prompt：发给该子 Agent 的具体指令（中文）
子 Agent 说明：
- data_agent：负责数据加载与概况查询。可完成「加载数据」「查看数据概况」等。
- training_agent：负责模型训练。可完成「训练贷款意愿模型与推荐模型」。
- recommendation_agent：负责推荐与意愿预测。可完成「为某客户推荐产品」「预测某客户对某产品的意愿」「查找相似产品」。
工作流程建议：用户说「加载数据」时可直接调用 load_data 或 Task(data_agent)；说「训练模型」时调用 train_models 或 Task(training_agent)；说「推荐」或「预测意愿」时调用 recommend/predict_propensity/similar_items 或 Task(recommendation_agent)。若用户未指定客户 ID 或产品 ID，可先调用 get_data_summary 了解数据规模并提示用户提供有效的 user_id 或 item_id。
回复时请先简要说明你执行了哪些工具或调用了哪个子 Agent，再给出结果摘要。"""
```

5.5.5 业务闭环

因为本例是用 Ollama 运行的 LLM。所以当用户输入自然语言时，会用户（Gradio 或命令行）输入一句自然语言。UI/CLI 调用 agents.runner.run_agent(msg)。Runner 用 core.config.build_agent_options_kwargs(cwd) 按当前后端组好 ClaudeAgentOptions，创建 ClaudeSDKClient，执行 connect() → query(msg) → receive_response()。SDK 驱动主 Agent 推理，主 Agent 可选择直接调用 MCP 或者委派 Task。主 Agent 汇总后，通过 receive_response() 流式 yield 消息。Runner 将消息 yield 给 UI/CLI，完成一次闭环。

整个时序图如下：



对应的源代码在 agents/runner.py 中的 run_agent()方法中，它可根据配置组好 Options（含 MCP、子 Agent），创建 SDK 客户端并执行 connect → query → receive_response，将流式消息逐条 yield 给 UI 或命令行。具体源代码如下：

```
async def run_agent(user_message: str) -> AsyncIterator[Any]:
    from claude_agent_sdk import ClaudeSDKClient, ClaudeAgentOptions
    cwd = str(PROJECT_ROOT)
    agent_kw = build_agent_options_kwargs(cwd=cwd)
    allowed = ["Read", "Grep", "Glob", "Bash", "Task"] + get_loan_agent_allowed_tools()
    mcp_server = build_loan_agent_mcp_server()
    sub_agents = build_sub_agent_definitions()
    options_dict = {
        **agent_kw,
        "system_prompt": MAIN_SYSTEM_PROMPT,
        "allowed_tools": allowed,
        "disallowed_tools": ["Write", "Edit"],
    }
    if mcp_server is not None:
        options_dict["mcp_servers"] = {"loan_agent": mcp_server}
    if sub_agents:
        options_dict["agents"] = sub_agents
    options = ClaudeAgentOptions(**options_dict)
    async with ClaudeSDKClient(options=options) as client:
        await client.connect()
        await client.query(user_message, session_id="default")
        async for message in client.receive_response():
            yield message
```

这个 Runner 的 run_agent()方法会被 2 个入口调用。其中一个是 Web UI 入口,即在第四个 Tab，题与说明文案引导用户输入自然语言，下方输入框与聊天区通过 run_chat_sync 绑定到 run_agent 的流式输出。

```
with gr.Tab(id=3, label="4. 智能对话"):
    gr.Markdown("用自然语言说「加载数据」「训练模型」「给客户 1 推荐 5 个产品」等，...")
    chat_input = gr.Textbox(placeholder="例如：先加载数据，再训练模型；或：给客户 1 推荐 5 个贷款产品", ...)
# 事件绑定：run_chat_sync 内用 asyncio 执行 run_agent，结果流式写入聊天框
```

另一个入口是命令行入口。即不启动 Web 时用 anyio 跑一次异步 main：向 run_agent 发送固定示例句，并把收到的每条 message 打印到控制台。

```
import anyio
from agents.runner import run_agent
async def main():
    async for message in run_agent(user_message="加载数据，然后训练模型。"):
        print(message)
if __name__ == "__main__":
```

```
    anyio.run(main)
```

5.5.6 LLM 除了 Ollama 模型以外，也可以切换到云端模型

在 core/config.py 中，LLM_BACKEND 决定用哪套后端。Ollama 时用 OLLAMA_* 与 OLLAMA_ENV（含 ANTHROPIC_BASE_URL）；Anthropic 时用 ANTHROPIC_MODEL，API Key 从环境变量读。

```
LLM_BACKEND = os.getenv("LLM_BACKEND", "ollama").strip().lower()
OLLAMA_BASE_URL = os.getenv("OLLAMA_BASE_URL", "http://localhost:11434")
OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "qwen2.5:32b")
OLLAMA_ENV = {
    "ANTHROPIC_AUTH_TOKEN": "ollama",
    "ANTHROPIC_API_KEY": "",
    "ANTHROPIC_BASE_URL": OLLAMA_BASE_URL,
}
ANTHROPIC_MODEL = os.getenv("ANTHROPIC_MODEL", "claude-sonnet-4-20250514")
```

根据 LLM_BACKEND 调用对应的 build_*_options_kwargs，返回 ClaudeAgentOptions 所需的 model、cwd、以及（Ollama 时）env 等参数字典。

```
def build_agent_options_kwargs(cwd: str | None = None) -> dict:
    """根据 LLM_BACKEND 返回对应后端的 ClaudeAgentOptions kwargs。"""
    if LLM_BACKEND == "anthropic":
        return build_anthropic_options_kwargs(cwd=cwd)
    return build_ollama_options_kwargs(cwd=cwd)
```

环境变量示例在.env.example 中。正式使用时需要将其重命名为.env 文件。

6 相关资源

https://github.com/charles-wang888/claude-agent-sdk-demo-retail-loan-ml